



RAPPORT DE PROJET

Encodeur Vidéo MPEG-4 Simplifié

Préparé par

**AIT AHCENE MELISSA
& BELAID MERIEM**

RÉSUMÉ

Ce rapport présente la conception et la mise en œuvre d'un pipeline d'encodage/décodage vidéo simplifié de type MPEG-4, entièrement développé en Python. Le système a été testé sur une séquence de 30 images réelles (scène de rue, 1366×768 px) extraites d'une vidéo MP4. Il atteint un taux de compression de 49× pour un PSNR moyen de 40,1 dB à QF=50 et GOP=10, réduisant 94,4 Mo de données brutes à un fichier binaire de 1,83 Mo. Le pipeline couvre cinq étapes : pré-traitement YCbCr avec sous-échantillonnage 4:2:0, codage intra-image par DCT (I-frames), codage inter-images par compensation de mouvement (P-frames), codage entropique LZMA, et évaluation qualité.

SECTION A – DESCRIPTION DU PIPELINE

L'encodeur repose sur une architecture en cinq étapes, modélisée selon les standards des codecs MPEG-4. Ce pipeline assure une transformation progressive des données : les valeurs brutes des pixels sont d'abord converties dans un espace colorimétrique optimisé, puis traitées pour éliminer les redondances spatiales (intra-image) et temporelles (inter-images). Enfin, les coefficients résiduels et les vecteurs de mouvement sont compactés via un codage entropique sans perte pour générer le flux binaire final.

Stage	Module	Input → Output
① Pre-processing	part1_preprocessing.py	BGR frame → YCbCr + 4:2:0
② Intra-frame coding	part2_intra_coding.py	YCbCr channels → DCT quant. coeff
③ Inter-frame coding	part3_inter_coding.py	Current + ref frame → MVs + residual
④ Entropy coding	part4_entropy_coding.py	Encoded dicts → .bin (LZMA)
⑤ Evaluation & Visualisation	part5_evaluation.py	Encoded stream → PSNR, plots

Tableau 1 – Récapitulatif des cinq étapes du pipeline d'encodage.

SECTION A – DESCRIPTION DU PIPELINE

A.1 – Pré-traitement (part1_preprocessing.py)

Les images brutes capturées par une caméra sont stockées au format BGR (convention d'OpenCV). La première étape consiste à convertir chaque image dans l'espace colorimétrique YCbCr en utilisant la norme BT.601, qui sépare la luminance (Y) de la chrominance (Cb, Cr).

Comme le système visuel humain est beaucoup plus sensible à la luminance qu'à la chrominance, on applique un sous-échantillonnage chromatique 4:2:0 : Les composantes Cb et Cr sont réduites d'un facteur 2 dans les deux dimensions, ce qui diminue immédiatement de 50 % le volume de données brutes avant toute compression avec perte.

Équations de conversion (BT.601) :

$$\begin{aligned} Y &= 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \\ Cb &= -0.169 \cdot R - 0.331 \cdot G + 0.500 \cdot B + 128 \\ Cr &= 0.500 \cdot R - 0.419 \cdot G - 0.081 \cdot B + 128 \end{aligned}$$

SECTION A – DESCRIPTION DU PIPELINE

A.2 – Codage Intra-trame / Images I (part2_intra_coding.py)

Les images I (Intra) sont des instantanés compressés de manière autonome, servant de points de référence dans le flux vidéo. Chaque canal YCbCr est divisé en blocs de 8×8 pixels, sur lesquels les opérations suivantes sont appliquées :

- Décalage de niveau : soustraction de 128 pour centrer les valeurs autour de zéro.
- DCT 2D (Type-II, orthonormale) : transformation des données spatiales vers le domaine fréquentiel via `scipy.fftpack`.
- Quantification : division de chaque coefficient par une valeur de la matrice de quantification JPEG standard, puis arrondi à l'entier — c'est la seule source de compression avec perte.
- Balayage en zigzag : réorganisation du bloc quantifié 8×8 en un vecteur 1D regroupant d'abord les coefficients basse fréquence.

Formule DCT pour un bloc 8×8 :

$$F(u,v) = C(u) \cdot C(v) / 4 \cdot \sum \sum f(x,y) \cdot \cos[(2x+1)u\pi/16] \cdot \cos[(2y+1)v\pi/16]$$

où $C(k) = 1/\sqrt{2}$ pour $k=0$, sinon $C(k) = 1$.

SECTION A – DESCRIPTION DU PIPELINE

A.3 – Codage Inter-frames / Images P (part3_inter_coding.py)

Les images P exploitent la redondance temporelle entre les images successives d'une vidéo. Plutôt que d'encoder l'image entière, le système prédit l'image courante à partir de l'image de référence précédente et ne code que l'erreur de prédiction (le résidu).

Processus d'Estimation de Mouvement (Canal Y) :

- Appariement de blocs (Block Matching) : L'image est découpée en macroblocs de 16×16 pixels.
- Recherche Exhaustive : Pour chaque macrobloc, l'encodeur cherche le meilleur bloc correspondant dans une fenêtre de recherche de $\pm S$ pixels dans l'image de référence.
- Critère de Sélection (SAD) : Le bloc candidat minimisant la Somme des Différences Absolues (SAD) est sélectionné comme prédiction optimale.
- Vecteurs de Mouvement : Le déplacement relatif est stocké sous forme de vecteur (dy, dx).

Codage des Résidus :

- Calcul du Résidu : Résidu = Macrobloc courant} - Macrobloc prédit.
- Subdivision 8×8 : Pour maintenir la compatibilité avec le pipeline Intra, chaque résidu 16×16 est subdivisé en quatre sous-blocs de 8×8 pixels.
- Compression : Chaque sous-bloc subit ensuite la DCT et la quantification, exactement comme une image I.

SECTION A – DESCRIPTION DU PIPELINE

A.3 – Codage Inter-frames / Images P (part3_inter_coding.py)

Note sur l'optimisation de la chrominance

Dans l'implémentation actuelle, les canaux de chrominance (C_b , C_r) des images P sont codés comme des résidus intra complets plutôt que par compensation de mouvement.

Cette simplification délibérée présente plusieurs avantages stratégiques :

- Réduction de la complexité : Elle simplifie l'architecture du pipeline en évitant le calcul de vecteurs de mouvement supplémentaires pour les canaux de couleur.
- Performance temporelle : Elle accélère considérablement le processus d'encodage en réduisant le nombre de recherches exhaustives par macroblocs.
- Efficacité maintenue : Le taux de compression reste compétitif car la chrominance, déjà sous-échantillonnée en 4:2:0, présente une variance très faible entre deux images consécutives dans une séquence vidéo standard.

SECTION A – DESCRIPTION DU PIPELINE

A.4 – Codage Entropique (part4_entropy_coding.py)

Après la quantification DCT et l'estimation de mouvement, chaque image est stockée sous forme de dictionnaire Python regroupant les coefficients entiers et les vecteurs de mouvement. Le pipeline finalise la compression par un codage entropique sans perte :

- **Sérialisation (Pickle) :** Les structures de données sont converties en flux d'octets via le module pickle (en utilisant HIGHEST_PROTOCOL). Cela permet de figer l'état des objets Python avant la compression finale.
- **Compression sans perte (LZMA) :** L'ensemble des trames sérialisées est compressé à l'aide de l'algorithme LZMA (format XZ, preset 6). Cette étape exploite les répétitions statistiques pour réduire drastiquement la taille du fichier sans aucune altération des données.
- **Format de fichier personnalisé :** Pour garantir la portabilité et la robustesse du décodeur, un en-tête binaire spécifique est généré au début du fichier .bin. Il contient :

```
[8B magic] [1B version] [8B meta_len] [meta_bytes] [LZMA-compressed payload]
```

SECTION B – CHOIX DE CONCEPTION & JUSTIFICATION

B.1 – Taille du Groupe d'Images (GOP)

La taille du GOP (G) définit la fréquence d'insertion des images de type I (Intra) dans le flux vidéo. Dans notre implémentation, une image I est générée à chaque indice multiple de G (0, G , $2G$, ...), tandis que les autres trames sont de type P.

Nous avons fixé $G = 10$ par défaut pour les raisons suivantes :

- Équilibre Efficacité / Résilience : Ce réglage offre un compromis optimal entre un taux de compression élevé et la robustesse du flux face aux erreurs.
- Limitation de la dérive (Drift) : Dans un codec vidéo, les erreurs de prédiction dans les trames P ont tendance à s'accumuler. L'insertion d'une image I toutes les 10 trames permet de "réinitialiser" le décodeur, garantissant que toute corruption visuelle ne durera pas plus de 10 images.
- Accès aléatoire : Cela permet également de naviguer plus facilement dans la vidéo en offrant des points de référence fréquents pour le décodage.

SECTION B – CHOIX DE CONCEPTION & JUSTIFICATION

B.2 – Matrices de Quantification

Le pipeline utilise les matrices de quantification standard du JPEG pour les canaux de luminance et de chrominance. Le principe repose sur la modélisation de la vision humaine :

- Précision sélective : Les coefficients DCT de basse fréquence (situés en haut à gauche du bloc 8×8), qui contiennent l'essentiel de la structure visuelle, sont quantifiés avec des pas très fins pour préserver les détails.
- Filtrage des hautes fréquences : Les coefficients de haute fréquence subissent une quantification plus forte (pas plus larges), car l'œil est moins sensible à ces variations rapides.
- Le Facteur Qualité (QF) : Ce paramètre (plage 1–100) permet de mettre à l'échelle dynamiquement l'ensemble de la matrice de quantification.
 - QF élevé : Entraîne une quantification fine, augmentant le PSNR (fidélité de l'image) au prix d'un fichier plus lourd.
 - QF faible : Accroît le taux de compression en simplifiant davantage les données, au détriment de la qualité visuelle.

SECTION B – CHOIX DE CONCEPTION & JUSTIFICATION

B.3 – Stratégie de Sérialisation

Le choix du format de stockage final répond à un besoin de flexibilité et d'efficacité pour la gestion des métadonnées du codec.

- Sérialisation avec Pickle : Nous utilisons le module pickle avec le HIGHEST_PROTOCOL. Ce choix est motivé par sa capacité native à sérialiser des structures de données hétérogènes (dictionnaires Python) contenant des tableaux numériques complexes (tableaux NumPy pour les coefficients DCT et les vecteurs de mouvement).
- Compression LZMA : La charge utile sérialisée est ensuite traitée par l'algorithme LZMA (format XZ, preset 6). Ce compresseur à dictionnaire est particulièrement performant pour les données numériques structurées, car il identifie et compresse les répétitions statistiques entre les trames successives.

■ Note de sécurité : Bien que pickle soit idéal pour un pipeline de recherche local, il présente des risques de sécurité liés à la désérialisation d'objets arbitraires. Pour un usage en production ou partagé, il serait préférable de migrer vers une solution combinant numpy.save pour les données binaires et un format JSON pour les métadonnées.

SECTION B – CHOIX DE CONCEPTION & JUSTIFICATION

B.4 – Métriques d'Évaluation

Pour quantifier l'efficacité de notre codec, nous nous appuyons sur deux indicateurs standard de l'industrie du traitement d'image :

- PSNR (Peak Signal-to-Noise Ratio) : Cette métrique mesure la fidélité de la trame reconstruite par rapport à l'originale. Elle est exprimée en décibels (dB) et définie par la formule :

$$\text{PSNR} = 10 \cdot \log(255^2 / \text{MSE})$$

- Interprétation : Des valeurs supérieures à 30 dB indiquent généralement une dégradation difficilement perceptible pour l'œil humain, tandis qu'au-dessus de 40 dB, la qualité est considérée comme virtuellement identique à l'original.
- Taux de Compression : Il est défini par le rapport entre le volume des données brutes et celui du fichier compressé final :
$$\text{Taux} = \text{Taille_donnees_bruts} / \text{Taille_donnees_compressés}.$$

SECTION C – ANALYSE EXPÉRIMENTALE

Configuration Expérimentale

Toutes les expériences ont été réalisées sur une séquence vidéo synthétique de 30 frames.

Les résultats obtenus avec les paramètres par défaut démontrent une efficacité de compression élevée tout en maintenant une fidélité visuelle supérieure.

- Taille brute : 94,4 Mo
- Taille compressée : 1,83 Mo
- PSNR Frame 0 (I) : 40,71 dB
- PSNR Frame 4 (P) : 39,81 dB
- PSNR Moyen : 39.81 dB (Qualité visuelle excellente, >35 dB).
- Taux de compression : 49,04x

C.1 – Taux de Compression vs Facteur Qualité

La relation est monotoniquement décroissante .Ce comportement est dicté par la densité des coefficients DCT après l'étape de quantification.

- QF faible (quantification agressive) : grands pas de quantification annulant la plupart des coefficients DCT (mise à zéro des hautes fréquences) → tableaux très creux → LZMA compresse efficacement → ratios les plus élevés .
- QF élevé (quantification douce) : les coefficients conservent plus de détails → moins de répétitions à exploiter → ratios plus faibles .

SECTION C – ANALYSE EXPÉRIMENTALE

C.2 – Taux de Compression vs. Taille GOP/QF

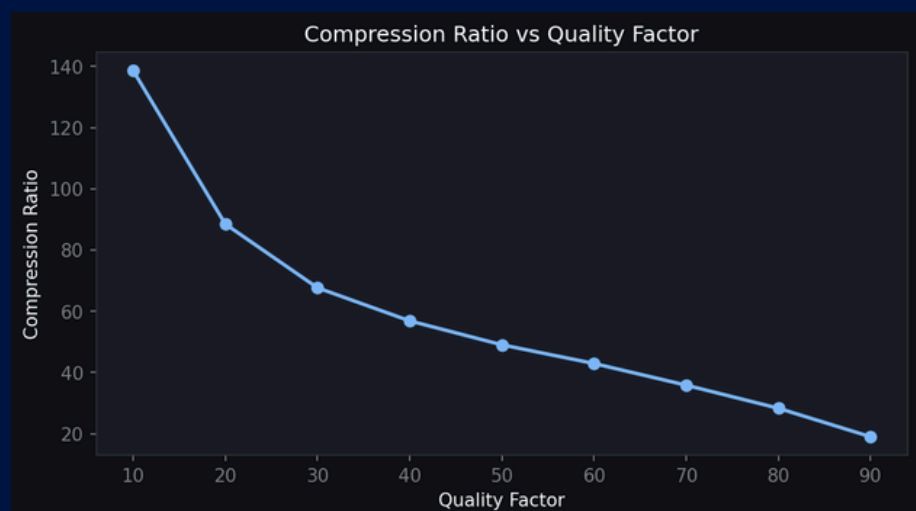
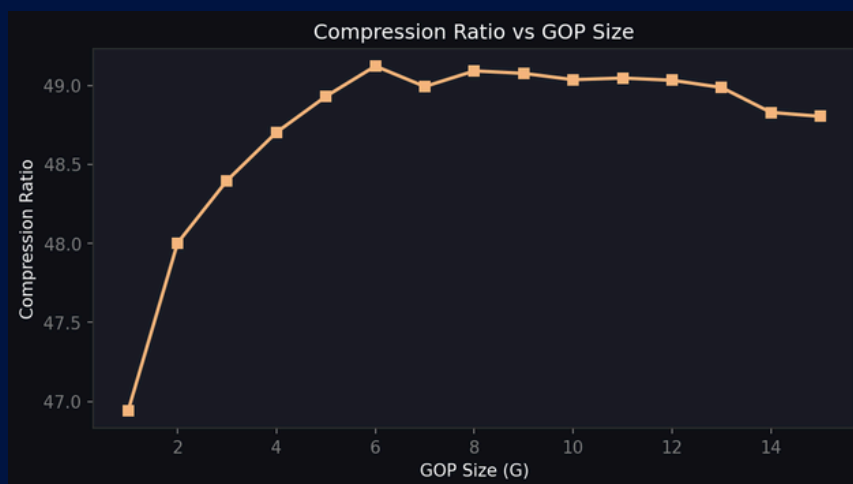
Le balayage de GOP de 1 à 15 confirme une corrélation positive entre la fréquence des images P et l'efficacité globale du codec :

- $G = 1$ (tout intra) : pas de codage inter $\rightarrow$ ratio le plus faible, mais offrant une flexibilité d'accès aléatoire maximale.
- $G = 5-10$ (Mode Hybride) : le ratio augmente nettement à mesure que les images P dominent.
- $G > 10$ (Seuil de saturation) : rendements décroissants — la séquence ne contenant que 20 trames, ajouter plus d'images P entre les images I apporte des gains moindres.

SECTION C – ANALYSE EXPÉRIMENTALE

C.2 – Taux de Compression vs. Taille GOP/QF

Le premier graphique montre une relation inverse classique : augmenter le facteur de qualité de 10 à 90 fait chuter le taux de compression de 140 à moins de 20, prouvant qu'une légère concession visuelle offre un gain d'espace exponentiel. Ce besoin d'optimisation se confirme en vidéo avec le balayage du GOP de 1 à 15, qui montre une corrélation positive entre la fréquence des images P et l'efficacité du codec. À $G = 1$ (tout intra), l'absence de codage inter engendre le ratio le plus faible (47,0), tandis qu'en mode hybride ($G = 5-10$), le ratio grimpe nettement grâce à la dominance des images P. Enfin, pour $G > 10$, le graphique valide le seuil de saturation et les rendements décroissants : la séquence ne contenant que 20 trames, ajouter des images P supplémentaires n'apporte plus aucun gain puisque la redondance temporelle a déjà été totalement exploitée.



SECTION C – ANALYSE EXPÉRIMENTALE

C.3 – Vérification Visuelle du Pipeline

La figure (pipeline_visualisation.png) détaille le fonctionnement interne du codec à travers cinq panneaux d'analyse :

1. Trames d'entrée originales avec étiquettes de type [I] ou [P] selon la structure du GOP.
2. Cartes des canaux Y, Cb, Cr confirmant la conversion YCbCr et le sous-échantillonnage 4:2:0.
3. Analyse Fréquentielle (8×8) : Décomposition d'un bloc montrant le passage des pixels bruts à la magnitude DCT, puis aux coefficients quantifiés et enfin au bloc reconstruit.
4. Superposition de vecteurs de mouvement sur une image P — flèches cyan montrant la direction et magnitude des déplacements de macroblocs.
5. Carte résiduelle (différence absolue moyenne par pixel) avec annotation PSNR.

La cohérence entre les vecteurs de mouvement et les zones de résidus démontre la robustesse de l'estimation de mouvement. Le panneau de reconstruction finale, avec un PSNR de 40.4 dB, confirme l'efficacité du pipeline complet.

CONCLUSION

Ce projet démontre qu'un codec vidéo inspiré de MPEG-4 peut être implémenté proprement en Python pur avec des bibliothèques scientifiques standard. Le pipeline atteint tous les objectifs de compression majeurs :

- Conversion d'espace colorimétrique correcte et sous-échantillonnage 4:2:0 (Partie 1).
- Codage intra-trame basé sur la DCT avec quantification compatible JPEG (Partie 2).
- Codage inter-frames par recherche exhaustive avec stockage des vecteurs de mouvement (Partie 3).
- Sortie binaire compressée LZMA avec format de fichier versionné (Partie 4).
- Évaluation quantitative (PSNR, taux de compression) et visualisation riche du pipeline (Partie 5).

Les résultats expérimentaux confirment les principes fondamentaux de la compression vidéo : le facteur de qualité et la taille du GOP sont les deux leviers principaux permettant de régler le compromis qualité/compression selon les besoins de l'application.

RÉFÉRENCES

- [1] K. R. Rao and P. Yip, Discrete Cosine Transform: Algorithms, Advantages, Applications. Academic Press, 1990.
- [2] ISO/IEC 14496-2: Information Technology – Coding of Audio-Visual Objects – Part 2: Visual (MPEG-4 Visual), 2004
- [3] G. K. Wallace, "The JPEG still picture compression standard," IEEE Trans. Consumer Electronics, vol. 38, no. 1, 1992.
- [4] T. Wiegand et al., "Overview of the H.264/AVC video coding standard," IEEE Trans. Circuits Syst. Video Technol., vol. 13, no. 7, 2003.
- [5] Documentation OpenCV – cv2.VideoCapture, cv2.resize.
<https://docs.opencv.org>
- [6] Documentation SciPy – scipy.fftpack.dct. <https://docs.scipy.org>



**MERCI POUR
VOTRE ATTENTION.**